

PROJECT MS · CHARACTER FRAMEWORK

신규 캐릭터 양산 가이드

프리팸 · 스크립트 · 전투 API · 네트워크 객체

전체 기능 개정 · 2026-08-18

CharacterBase 기준 · Unity 6000.3.11f1 · Photon Fusion Shared Mode

0. 이 문서를 보는 순서

처음 캐릭터를 만드는 경우에는 제작 순서를 따라가고, 필요한 기능만 추가하는 경우에는 해당 항목으로 바로 이동합니다.

작업별 확인 위치

하고 싶은 작업	먼저 볼 곳
새 캐릭터 처음부터 제작	1 → 3 → 4 → 5 → 6
공통 프리랩 필수 구성 확인	3, 4
평타·QE·궁극기 작성	6, 7, 9
이동·대시·슬로우·넉백	8
상대 입력 감지·행동 봉인·조준 방해	8.1, 8.2, 8.3
총알·화살 등 직선 투사체	10, 11
소유 오브젝트 공통 구조와 이유	13
설치물·SPARK Q 형태 노드·포탑	14, 14.1, 14.2
이동·공격하는 소환체	15
수류탄·섬광탄·연막탄	16
조화·제거·개수 제한과 HP	17, 18
자동 검사와 Fusion 2인 테스트	20, 21

공통 규칙

- 설치물·소환체·투척체는 BASE 공통 API로 생성하고 제거합니다.
- 캐릭터 스크립트에서 Runner.Spawn, Runner.Despawn, [Networked], [Rpc]를 직접 추가하기 전에 공통 API를 확인합니다.
- 구현 후 Validator와 Fusion 2인 Shared 테스트를 통과시킵니다.

1. 한눈에 보는 제작 순서

순서	작업	만들어지는 결과
1	이름과 폴더를 정한다	캐릭터 전용 작업 공간
2	CharacterTemplate.cs 를 복사한다	{Name}Character.cs
3	CharacterDefinition 을 만든다	체력, 이동, 데미지, 쿨다운 데이터
4	CharacterTemplate.prefab 을 원본으로 연결형 프리팹을 만든다	{Name}Character.prefab
5	루트 컴포넌트와 자식 계층을 연결한다	이동과 네트워크가 되는 캐릭터
6	평타 하나를 구현한다	공격과 쿨다운 검증
7	Q, E, 궁극기를 하나씩 구현한다	캐릭터 전투 규칙
8	투사체, 설치물, 소환체, 투척체를 필요한 만큼 만든다	스킬용 네트워크 프리팹
9	비주얼, 사운드, 패시브를 연결한다	화면 표현 완성
10	자동 검사 도구와 Fusion 2인 테스트를 실행한다	양산 완료 판정

완료 산출물

01.Script/Character/Characters/{Name}/{Name}Character.cs
 03.Prefabs/Character/{Name}/{Name}Character.prefab
 10.Data/Character/{Name}/{Name}CharacterDefinition.asset
 10.Data/Character/{Name}/{Name}VisualProfile.asset # 필요한 경우
 03.Prefabs/Character/{Name}/{Name}{Skill}Projectile.prefab
 03.Prefabs/Character/{Name}/{Name}{Skill}Entity.prefab

2. 폴더와 이름 규칙

신규 캐릭터 표준

종류	규칙	예시
폴더	{Name} PascalCase	Spark, Chaser, Gunner
클래스	{Name}Character	SparkCharacter
메인 프리팹	{Name}Character.prefab	SparkCharacter.prefab
수치 데이터	{Name}CharacterDefinition.asset	SparkCharacterDefinition.asset
비주얼 데이터	{Name}VisualProfile.asset	SparkVisualProfile.asset
직선 투사체	{Name}{Skill}Projectile.prefab	SparkBasicProjectile.prefab
설치물/소환체	{Name}{Skill}Entity.prefab	SparkNodeEntity.prefab
물리 투척체	{Name}{Skill}Throwable.prefab	GunnerGrenadeThrowable.prefab

대문자 전체 이름(**SPARK**), 축약형(**Char**), 입력 키 이름(**Q**)은 표시 이름이나 임시 파일에만 사용한다. 클래스와 최종 프리팹은 역할이 드러나는 이름을 사용한다.

기존 자산 주의

현재 CHASER, SPARK, Gunner 프리팹은 신규 표준이 정해지기 전에 서로 다른 방식으로 제작된 기존 자산이다. 프리팹 계층, 필수 컴포넌트, 설정 파일 연결 방식이 신규 표준과 다를 수 있으므로 새 캐릭터의 복제 원본으로 사용하지 않는다.

신규 캐릭터는 반드시 **03.Prefabs/Character/Common/CharacterTemplate.prefab**을 원본으로 사용한다. 이렇게 해야 공통 계층과 네트워크 컴포넌트를 물려받으면서 캐릭터별로 달라지는 스프라이트, 충돌 크기, 수치 설정만 따로 관리할 수 있다.

기존 캐릭터를 수정할 때는 동작 중인 구조를 한 번에 신규 표준으로 바꾸지 않는다. 먼저 현재 프리팹과 스크립트의 연결 상태를 확인하고, 구조 이전이 필요하다면 캐릭터별 작업과 공통 템플릿 변경을 분리해 검토한다.

3. 기능을 만들기 전에 공통 기반 코드 선택하기

기능	사용할 공통 기반/API	직접 작성하지 않는 것
원형·부채꼴·선·사각형 공격	FindDamageables... + DealDamage	Physics2D 검색 뒤 HP 직접 변경
총알·화살·직선 마법탄	CharacterProjectile + SpawnProjectile	Runner.Spawn 직접 호출
노드·지뢰·장판·포탑	CharacterDeployable + SpawnOwnedEntity	맵 구조물 기반 사용
펫·분신·드론	CharacterSummon + SpawnOwnedEntity	소유권·제거 코드 재작성
수류탄·섬광탄·연막탄	CharacterThrowable + SpawnThrowable	직선 투사체에 중력 추가
돌진	StartDash	Rigidbody 속도 직접 변경
지연 실행·시간 종료	ScheduleTimer	Invoke-Coroutine-로컬 시간
이동 속도 감소	ApplySlow	이동 속도 값을 직접 덮어쓰기
스킬 오브젝트 개수 제한	SpawnOwnedEntity(..., maxCount)	별도 목록 검색과 직접 제거

처음에는 행동 종류(context.Action)를 전달하는 간단 생성 함수를 사용합니다. 한 행동에서 서로 다른 그룹을 따로 관리하거나 세부 실패 이유가 필요할 때만 요청 구조체를 사용합니다.

4. 캐릭터 프리팹 표준 구조

```

(Name)Character          # 프리팹 루트
├─ Rigidbody2D
├─ Collider2D
├─ NetworkObject
├─ NetworkRigidbody2D
├─ (Name)Character      # CharacterBase 파생 스크립트
├─
├─ VisualRoot
├─   └─ CharacterVisualController
├─   └─ Body
├─     └─ SpriteRenderer
├─     └─ Animator      # 선택
├─   └─ AimVisualRoot   # 선택
├─   └─ DustEffect      # 선택
├─
├─ Sockets
├─   └─ AttackOrigin
├─   └─ ProjectileOrigin # 투사체 캐릭터만
├─   └─ WeaponRoot      # 무기 표현이 있을 때
├─   └─ EffectOrigin     # 액션 VFX가 있을 때
├─
├─ WorldCanvas          # 선택
├─   └─ HealthBar

```

루트에는 게임 판정과 네트워크 컴포넌트, 자식에는 화면 표현과 위치 기준점을 둔다. **CharacterVisualController**는 캐릭터마다 여러 개 만들지 않고 하나만 둔다.

4.1 루트 필수 컴포넌트

컴포넌트	필수 설정	이유
{Name}Character	Definition, Visual, Sockets 연결	입력, 행동, 체력, 쿨다운 실행
NetworkObject	루트에 1개	Fusion 네트워크 객체 식별
Rigidbody2D	게임 물리에 맞게 Body Type/중력 설정	이동과 낙백 처리
Collider2D	캐릭터 몸 크기에 맞춤	지면, 벽, 피격 판정
NetworkRigidbody2D	루트에 1개	위치와 물리 동기화

금지 조합

- NetworkRigidbody2D와 NetworkTransform을 동시에 붙이지 않는다.
- CharacterTemplate와 {Name}Character를 동시에 붙이지 않는다.
- 자식 오브젝트에 두 번째 NetworkObject를 붙이지 않는다. 별도 네트워크 객체라면 별도 프리팹으로 분리한다.
- 캐릭터 루트에 스킬별 투사체나 설치물 컴포넌트를 붙이지 않는다.

캐릭터 설정 데이터와 화면 표현 연결

{Name}Character의 Definition 칸에는 해당 캐릭터의 수치 설정 파일(CharacterDefinition)을 연결한다. Visual 칸에는 자식 VisualRoot의 화면 표현 제어기(CharacterVisualController)를 연결한다. Input Camera는 보통 비워 두며 로컬 캐릭터가 Camera.main을 사용한다.

4.2 화면 표현과 위치 기준점 연결

CharacterVisualController 필수값

Unity Inspector에 보이는 칸	연결 대상
Body	크기 변형과 좌우 반전의 기준 Transform
Body Renderer	Body 아래 대표 SpriteRenderer
Profile	공통 또는 캐릭터별 CharacterVisualProfile
Aim Visual Root	조준 회전/반전할 무기 또는 팔, 없으면 비움

공격 위치 기준점(Gameplay Sockets)

Socket	용도	비어 있으면
Attack Origin	근접/범위 공격 중심	캐릭터 루트
Projectile Origin	총알, 화살 생성 위치	Attack Origin
Weapon Root	화면 무기 부모	null
Effect Origin	액션 VFX 생성 위치	Attack Origin

위치 기준점(Socket)은 컴포넌트가 아니라 캐릭터 스크립트의 Unity Inspector 안에 있는 참조 묶음이다. 빈 GameObject를 만든 뒤 루트의 **{Name}Character > Gameplay Sockets** 칸에 끌어 놓는다.

4.3 원본 연결형 프리팹 만드는 순서

원본 연결형 프리팹은 공통 템플릿과 연결을 유지하면서 캐릭터별 차이만 저장하는 프리팹이다. Unity 메뉴에서는 이를 **Prefab Variant**라고 표시한다.

1. **03.Prefabs/Character/Common/CharacterTemplate.prefab**을 선택한다.
2. 우클릭 메뉴에서 **Create > Prefab Variant**를 실행한다.
3. **{Name}Character.prefab**으로 이름을 바꾸고 해당 캐릭터 폴더에 저장한다.
4. 프리팹 편집 화면(Prefab Mode)에서 임시 **CharacterTemplate** 컴포넌트를 제거한다.
5. **{Name}Character** 스크립트를 추가한다.
6. 수치 설정 파일(Definition), 화면 표현 제어기(Visual), 위치 기준점(Sockets)을 연결한다.
7. Body 스프라이트, Collider 크기, Animator, VFX, 사운드를 캐릭터별로 설정한다.
8. 변경 사항(**Overrides**) 목록을 검토하고 캐릭터별로 바꾼 항목만 남긴다.

원본 연결형 프리팹 작업 규칙

- 캐릭터 프리팹에서 **Apply All to Base**(모든 변경을 원본에 적용)를 누르지 않는다. 다른 캐릭터까지 함께 바뀔 수 있다.
- 공통 구조 변경이 필요하면 공통 기반 코드 담당자에게 요청하고 **CharacterTemplate.prefab**을 별도 커밋으로 수정한다.
- 공통 템플릿의 자식을 삭제하기보다 사용하지 않는 선택 자식은 비활성화하거나 참조를 비운다.
- 원본 연결이 끊긴 일반 프리팹 복사본을 양산 기준으로 사용하지 않는다.

5. 캐릭터 수치 설정 파일(CharacterDefinition)

Create > Project MS > Character > Definition으로 생성한다.

그룹	주요 값	작업 기준
Identity	Display Name	UI 표시 이름
Stats	Max Health, 행동별 Damage	실제 기본 수치만 저장
Cooldowns	평타/Q/E/Dash/Ultimate	성공한 행동 뒤 적용
Knockback	Base Force, Duration, Upward Bias	피격 넉백과 히트스턴
Movement	Move Speed, Acceleration, Jump, Ground Layer	공통 이동 모듈 설정
Dash	Default Dash Power/Duration	기본 대시 설정
Ultimate Gauge	Uses Gauge, Max, Per Damage	게이지형 궁극기
Auto Hop	사용 여부, 주기, 힘, 입력 임계값	자동 통통 이동
Input	키 바인딩	현재 프로젝트 기본 입력

현재 체력, 일시 강화 효과, 잔탄, 남은 재사용 대기시간은 설정 파일(asset)에 저장하지 않는다. 플레이 중 변하는 값은 **CharacterBase**가 관리한다.

Ground Layer가 비어 있으면 지면 판정과 점프가 정상 동작하지 않는다. 넉백 Duration 동안 이동과 행동이 잠기는 히트스턴이 적용된다.

5.1 비주얼과 사운드 설정

CharacterVisualProfile

Create > Project MS > Character > Visual Profile

- 점프 Stretch, 착지 Squash
- 공중 속도에 따른 늘어남
- Idle Wobble
- 피격 색상과 피격 반동
- 장식 Jiggle의 강성, 감쇠, 최대 이동과 회전

CharacterVisualController 선택값

기능	Unity Inspector 설정
조준 방향	Aim Visual Mode, Aim Visual Root, Rotation Offset
이동 먼지	Dust Effect
애니메이션	Animator Trigger: Jump, Land, Damaged, Dead, Revived, 행동 이름
행동 VFX	Action Effects에 Action, Prefab, Lifetime, Angle Offset
카메라 흔들림	Damaged Shake Magnitude/Duration
공통 사운드	CharacterCommonSoundSet
행동 사운드	Action Sounds에 Action, Clip, Volume
장식 흔들림	Jiggle Targets
좌우 반전 보정	Counter Flip Targets

VFX와 사운드는 게임 판정에 사용하지 않는다. 원격 플레이어 화면에서도 필요한 액션 VFX는 **PlayActionEffect** 경로를 사용한다.

6. 캐릭터 스크립트 시작하기

```
using ProjectMS.CharacterSystem;

public sealed class NovaCharacter : CharacterBase
{
    protected override bool OnBasicAttack(CharacterActionContext context)
    {
        // 공격을 실제로 실행했으면 true
        return true;
    }

    protected override bool OnSkillQ(CharacterActionContext context) => false;
    protected override bool OnSkillE(CharacterActionContext context) => false;
    protected override bool OnUltimate(CharacterActionContext context) => false;

    protected override void OnResetCharacter()
    {
        // 캐릭터별 로컬 상태 초기화
    }
}
```

반환 규칙

- **true**: 행동 성공. 잔탄 차감, 자동 쿨다운, 행동 이벤트와 표현이 진행된다.
- **false**: 행동 취소. 프리팹 누락, 대상 없음, 조건 불충족 등에 사용하며 잔탄과 쿨다운이 변하지 않는다.
- **OnDash**를 구현하지 않으면 공통 기본 대시가 실행된다.

CharacterActionContext의 **Origin**, **AimDirection**, **AimWorldPosition**, **Damage**, **AimAngle**을 사용한다. 마우스 위치와 Definition 데미지를 캐릭터 스크립트에서 다시 계산하지 않는다.

7. 행동, 잔탄, 쿨다운

목적	API
행동 활성화/비활성	SetActionEnabled , IsActionEnabled
잔탄 설정/증감/조회	SetActionCharges , AddActionCharges , GetActionCharges
기본 쿨다운 변경/복원	SetCooldownDuration , ResetCooldownDuration
성공 뒤 자동 쿨다운	SetAutoCooldown
수동 쿨다운 시작/해제/조회	StartCooldown , ClearCooldown , GetCooldownRemaining

```
protected override void OnCharacterSpawned()
{
    SetActionCharges(CharacterActionType.SkillQ, 2);
    SetAutoCooldown(CharacterActionType.SkillQ, false);
}

protected override bool OnSkillQ(CharacterActionContext context)
{
    if (GetActionCharges(CharacterActionType.SkillQ) <= 0)
        return false;

    // 스킬 실행
    StartCooldown(CharacterActionType.SkillQ);
    return true;
}
```

StartCooldown(action)은 캐릭터 수치 설정 파일 또는 캐릭터별 덮어쓰기 값(Override)의 시간을 사용한다.
StartCooldown(action, seconds)는 이번 한 번만 지정 시간을 사용한다. 잔탄 **-1**은 무제한이다.

8. 이동, 대시, 슬로우, 히트스턴, 넉백

대시

- 일반 대시: **OnDash**를 구현하지 않는다.
- 특수 대시: **StartDash(context.AimDirection, power, duration)**을 호출한다.
- 이동이 잠긴 상태에서는 새 대시가 시작되지 않고 진행 중인 대시도 취소된다.

이동 잠금과 슬로우

```
SetMovementEnabled(false);
ApplySlow(target, 0.4f, 2f);
```

SetMovementEnabled(false)는 수평 이동, 점프, 자동 점프, 대시를 막는다. 수평 속도는 0이 되지만 중력과 낙하는 계속된다. 슬로우는 더 강한 값이 우선하며 같은 강도면 시간을 갱신한다.

넉백과 히트스턴

피해가 실제 적용되면 Definition의 Knockback 설정에 따라 공통 넉백과 히트스턴이 처리된다. 캐릭터별 스크립트에 서 피격자의 Rigidbody를 다시 밀지 않는다.

값	의미
Knockback Base Force	피해량과 무관한 기본 힘
Knockback Duration	넉백과 입력 잠금 시간
Knockback Upward Bias	위쪽으로 띄우는 비율

현재 상태는 **IsHitstunned**, **IsSlowed**, **SlowRatio**, **FacingDirection**, **AimDirection**으로 읽는다.

8.1 상대 입력 감지

실제 키 이름이 아니라 캐릭터에 연결된 기능 이름을 감지합니다. 키 배치가 바뀌어도 Jump, BasicAttack 같은 코드는 바꾸지 않습니다.

감지할 수 있는 입력

확인할 입력	사용할 값
점프를 누른 순간	CharacterInputType.Jump
기본 공격	CharacterInputType.BasicAttack
Q / E 스킬	CharacterInputType.SkillQ / SkillE
대시 / 궁극기	CharacterInputType.Dash / Ultimate
좌우 이동	GetObservedMoveDirection(target)
점프를 누르고 있음	IsInputHeld(target, CharacterInputType.Jump)

버튼을 누른 순간 감지

```
private bool watchingEnemyInput;

protected override void OnPassiveTick(float deltaTime)
{
    if (!watchingEnemyInput)
        return;

    CharacterBase enemy = All.Find(character =>
        character != null && character != this &&
        character.DamageTeamId != DamageTeamId);
    if (enemy != null &&
        WasInputPressed(enemy, CharacterInputType.Jump))
    {
        // 상대가 점프를 입력했을 때 실행할 처리
    }
}
```

WasInputPressed는 새 입력 한 번에 한 번만 true입니다. 감지를 시작한 뒤 매 네트워크 틱 호출합니다. 입력은 봉인 적용 전에 기록되므로, 봉인된 상대가 키를 눌러도 감지는 되지만 행동은 실행되지 않습니다.

8.2 상대 행동 봉인

필요한 조작만 골라 일정 시간 동안 막습니다. 각 조작의 남은 시간은 따로 관리됩니다.

봉인 종류

값	막는 조작
Movement	좌우 이동
Jump	점프 입력과 점프 유지
BasicAttack	기본 공격
SkillQ / SkillE	Q / E 스킬
Dash / Ultimate	대시 / 궁극기
AllActions	모든 공격과 스킬. 이동·점프는 허용
All	이동, 점프, 공격, 스킬 전체

사용 예제

```
// 점프만 2초 동안 막기
ApplyControlSeal(target, CharacterControlType.Jump, 2f);

// 공격과 스킬을 3초 동안 모두 막기
ApplyControlSeal(target, CharacterControlType.AllActions, 3f);

// 좌우 이동과 점프만 함께 막기
ApplyControlSeal(
    target,
    CharacterControlType.Movement | CharacterControlType.Jump,
    1.5f);
```

같은 봉인이 다시 들어오면 현재 남은 시간보다 긴 요청만 적용합니다. 3초가 남은 봉인에 1초 봉인이 들어와도 3초가 유지됩니다.

자기 자신, 아군, 죽은 대상 또는 0초 이하 요청은 공통 검증에서 무시됩니다. 요청과 시간은 대상 캐릭터의 네트워크 상태로 동기화됩니다.

8.3 조준 방해와 조준선 UI

실제 발사 방향을 바꾸는 기능과 화면의 조준선만 바꾸는 기능을 구분해서 사용합니다. 운영체제의 마우스 포인터 위치는 강제로 움직이지 않습니다.

기능 구분

API	바뀌는 것
ApplyAimInversion	실제 조준·발사 방향을 마우스 반대로 변경
ApplyAimAngleOffset	실제 조준·발사 방향을 지정 각도만큼 회전
SetCrosshairOffset	조준선 UI 위치만 픽셀 단위로 이동
SetCrosshairVisible	조준선 UI 표시 또는 숨김
SetSystemCursorVisible	기본 마우스 커서 표시 또는 숨김

사용 예제

```
// 실제 조준 방향을 반대로 적용
ApplyAimInversion(target, 2f);

// 실제 조준 방향을 반시계 방향으로 30도 회전
ApplyAimAngleOffset(target, 30f, 2f);

// 기본 커서를 숨기고 공통 조준선을 표시
ReplaceCursorWithCrosshair(target, 3f);

// 조준선 UI만 오른쪽 100, 아래 50픽셀 이동
SetCrosshairOffset(target, new Vector2(100f, -50f), 3f);
```

조준 반전과 각도 변경은 context.AimDirection, context.AimWorldPosition, AimDirection과 투사체 발사에 함께 적용됩니다. 조준선은 대상 플레이어의 로컬 화면에만 만들어지며 네트워크 오브젝트로 생성하지 않습니다.

효과는 죽음, 라운드 리셋, 디스폰 또는 지정 시간 종료 시 해제됩니다. 시스템 커서는 효과를 적용하기 전 표시 상태로 돌아갑니다.

9. 궁극기 게이지

Definition의 **Ultimate Uses Gauge**를 켜면 궁극기는 쿨다운 대신 게이지로 동작한다.

1. 적에게 실제 적용된 데미지만큼 게이지가 증가한다.
2. 증가량은 **AppliedDamage * UltimateGaugePerDamageDealt**다.
3. **UltimateGaugeMax**에 도달해야 궁극기 입력이 실행된다.
4. 궁극기 함수가 **true**를 반환하면 게이지가 0으로 초기화된다.
5. 게이지 모드에서는 Ultimate Cooldown을 사용하지 않는다.

읽기 전용 상태

```
bool gaugeMode = IsUltimateGaugeMode;  
float current = UltimateGaugeCurrent;  
float max = UltimateGaugeMax;
```

게이지를 캐릭터 스크립트의 별도 float로 저장하거나 RPC로 동기화하지 않는다. 설치물과 소환체가 준 피해도 캐릭터 소유 오브젝트(Owned Entity)의 공통 데미지 경로를 사용하면 게이지에 반영된다.

10. 근접과 범위 공격

공격 모양	조회 API
원형	FindDamageablesInCircle
회전 사각형	FindDamageablesInBox
폭이 있는 직선	FindDamageablesInLine
부채꼴	FindDamageablesInArc

```
protected override bool OnBasicAttack(CharacterActionContext context)
{
    var targets = FindDamageablesInArc(
        AttackOrigin.position,
        context.AimDirection,
        range,
        angle,
        targetLayer);

    foreach (IDamageable target in targets)
        DealDamage(target, context.Damage, CharacterDamageSource.Direct);

    return true; // 공격 모션과 판정을 실행했으므로 빗나가도 성공
}
```

FindEnemies...는 캐릭터만 필요한 기존 코드에 사용한다. 설치물과 소환체도 맞아야 하는 신규 공격은 **FindDamageables...**를 우선 사용한다. 대상의 HP를 직접 수정하지 않는다.

11. 직선 투사체 프리팹

```
{Name}{Skill}Projectile
├─ NetworkObject
├─ CharacterProjectile
├─ Collider2D          # 통과형 충돌(Is Trigger) 권장
├─ Visual
└─ SpriteRenderer 또는 VFX
```

CharacterProjectile은 직접 위치를 이동시키므로 **NetworkTransform**과 **NetworkRigidbody2D**를 추가하지 않는다.

Unity Inspector 항목	설정 기준
Lifetime	아무것도 맞지 않았을 때 제거 시간
Pierce Characters	캐릭터 관통 여부, 같은 대상은 1회만 피격
Wall Layer	벽과 지형 Layer
Collision Skin Width	고속 이동 충돌 여유, 기본 0.03
Projectile Angle Offset	스프라이트가 +X면 0, +Y면 -90
Hit Vfx Prefab	충돌 위치에 표시할 로컬 VFX
Hit Vfx Angle Offset	표면 방향 기준 보정
Hit Vfx Surface Offset	벽 안에 묻히지 않는 거리

Collider가 Target Layer 또는 Wall Layer와 실제로 충돌하도록 Physics Layer Matrix도 확인한다.

11.1 투사체 생성과 종료

```
[SerializeField] private CharacterProjectile projectilePrefab;
[SerializeField] private float projectileSpeed = 14f;
[SerializeField] private LayerMask targetLayer;

protected override bool OnBasicAttack(CharacterActionContext context)
{
    CharacterProjectile projectile = SpawnProjectile(
        projectilePrefab,
        ProjectileOrigin.position,
        context.AimDirection,
        projectileSpeed,
        context.Damage,
        targetLayer,
        skillId: 1001);

    return projectile != null;
}
```

규칙

- **CharacterProjectile.Initialize**을 직접 호출하지 않는다.
- **SpawnProjectile** 반환값으로 생성 성공을 확인한다.
- 같은 종류의 여러 투사체를 구분할 때 Layer를 늘리지 말고 반환 참조 또는 **SkillId**를 사용한다.
- 수동 종료는 **DespawnProjectile(projectile)**을 사용한다.
- 종료 이유는 **HitCharacter**, **HitOwnedEntity**, **HitWall**, **LifetimeExpired**, **Manual**이다.
- 관통 투사체도 캐릭터 소유 오브젝트(Owned Entity)나 벽에 맞으면 종료된다.

12. 데미지, 패시브, 콜백

데미지 흐름

공격 조회/충돌
 -> DamageRequest 생성
 -> 대상 CanReceiveDamage
 -> 공격자 ModifyOutgoingDamage
 -> 대상의 상태 변경 권한(State Authority)을 가진 플레이어에서 실제 체력 감소
 -> AppliedDamage 확정
 -> 게이지와 공격자/피격자 콜백

RequestedDamage와 **AppliedDamage**를 구분한다. 체력이 5 남은 대상에게 20을 요청하면 실제 적용량은 5다.

훅	사용 시점
ModifyOutgoingDamage	치명타, 후방 보너스 등 공격자 보정
OnDamaged(CharacterDamageInfo)	피격 반응, 공격자/스킬/위치 확인
OnDamageDealt	캐릭터에게 실제 적용된 데미지 처리
OnDamageableDealt	캐릭터와 캐릭터 소유 오브젝트를 함께 처리
OnOwnedEntityDamageDealt	대상이 사라져도 NetworkId 기반 결과 처리
OnProjectileDespawned	투사체 종료 이유별 후속 처리
OnDied	사망 시 캐릭터별 정리

패시브는 **Update**를 새로 만들기보다 **OnPassiveTick**, **OnDamaged**, **OnDamageDealt**, **OnJumped**, **OnLanded**, **OnSkillExecuted** 중 맞는 훅을 사용한다.

13. 캐릭터 소유 오브젝트 제작 흐름

설치물·소환체·물리 투척체는 생성 후 필드에 남아 여러 네트워크 규칙을 공유하므로 `CharacterOwnedEntity`에서 공통 관리합니다.

공통 기반에서 관리하는 이유

관리 항목	공통으로 처리하는 내용
생성자와 소유자	누가 생성했고 어느 캐릭터가 소유하는지 기록
팀	어느 팀의 오브젝트인지 기록하고 적·아군 판정에 사용
개수 제한	최대 유지 개수와 초과 시 교체 또는 생성 거부 정책
HP와 수명	피해로 파괴되거나 제한시간 종료 시 제거
소유자 상태	소유자 사망·퇴장 시 제거 또는 남은 시간 유지
피해 판정	자가·아군·적 피해 규칙과 실제 적용 피해 처리
네트워크 동기화	모든 클라이언트에서 생성과 제거 상태를 동일하게 유지
제거 결과	제거 사유 기록과 파괴 후 콜백 실행

제작 순서

순서	작업
1	설치물·소환체·물리 투척체 중 기반 클래스를 고릅니다.
2	프리팹 루트에 <code>NetworkObject</code> 와 파생 스크립트를 추가합니다.
3	HP, 수명, 충돌 레이어와 동기화 컴포넌트를 설정합니다.
4	캐릭터 스킬에서 공통 생성 API를 호출합니다.
5	고유 행동은 프리팹 파생 스크립트에 작성합니다.

14. 설치물 생성

설치물 프리팹은 `CharacterDeployable` 파생 스크립트를 사용합니다. 고정 위치 설치와 던진 뒤 설치를 먼저 구분합니다.

프리팹 루트 구성

구성	고정 위치 설치	던진 뒤 설치
필수	NetworkObject, 파생 스크립트	NetworkObject, 파생 스크립트
충돌	필요한 Collider2D	Trigger가 아닌 Collider2D
이동 동기화	NetworkTransform	Dynamic Rigidbody2D + NetworkRigidbody2D
생성 값	position	position + initialVelocity

고정 위치 설치 코드

```
[SerializeField] private MyDeployable deployablePrefab;

protected override bool OnSkillQ(CharacterActionContext context)
{
    MyDeployable deployable = SpawnOwnedEntity(
        deployablePrefab,
        context.Action,
        context.AimWorldPosition,
        maxCount: 1);

    return deployable != null;
}
```

캐릭터 스크립트는 생성 위치와 최대 개수만 결정합니다. 공격·감지·연결·폭발 같은 행동은 설치물 파생 스크립트가 처리합니다.

14.1 설치물 예제: SPARK Q 형태의 전기 노드

노드를 캐릭터 앞에서 던져 생성하고 최대 2개를 유지하는 예제입니다. 바닥 고정, 두 노드 연결과 선 데미지는 노드 스크립트에 추가합니다.

프리팹: NetworkObject + SparkQNode + Dynamic Rigidbody2D + Collider2D + NetworkRigidbody2D

```
[SerializeField] private SparkQNode nodePrefab;
[SerializeField] private float nodeThrowSpeed = 10f;

protected override bool OnSkillQ(CharacterActionContext context)
{
    SparkQNode node = SpawnOwnedEntity(
        nodePrefab,
        context.Action,
        ProjectileOrigin.position,
        maxCount: 2,
        initialVelocity: context.AimDirection * nodeThrowSpeed);

    return node != null;
}
```

```
public sealed class SparkQNode : CharacterDeployable
{
}
```

maxCount: 2이면 세 번째 노드를 만들 때 가장 오래된 노드가 제거됩니다. **context.Action**이 Q로 만든 노드를 같은 그룹으로 관리합니다.

14.2 설치물 예제: 일정 시간마다 공격하는 포탑

캐릭터는 포탑을 생성하고, 대상 탐지와 공격은 포탑 스크립트가 처리합니다.

```
[SerializeField] private SimpleTurret turretPrefab;

protected override bool OnSkillE(CharacterActionContext context)
{
    return SpawnOwnedEntity(
        turretPrefab, context.Action, context.AimWorldPosition) != null;
}
```

```
public sealed class SimpleTurret : CharacterDeployable
{
    [SerializeField] private float attackInterval = 1f;
    [SerializeField] private float attackRadius = 4f;
    [SerializeField] private float damage = 10f;
    [SerializeField] private LayerMask targetLayer;

    public override void FixedUpdateNetwork()
    {
        base.FixedUpdateNetwork();

        if (!TryUseInterval(attackInterval))
            return;

        IDamageable target = FindFirstDamageableInCircle(
            transform.position, attackRadius, targetLayer);

        if (target != null)
            DealDamage(target, damage);
    }
}
```

`TryUseInterval`은 공격 간격을 관리하고, `FindFirstDamageableInCircle`은 범위 안의 대상을 찾습니다. `DealDamage`는 공통 피해 처리 경로를 사용합니다.

15. 소환체 생성과 프리팹 구현

소환체 프리팹은 `CharacterSummon` 파생 스크립트를 사용합니다. 생성 위치와 최대 개수는 캐릭터가 정하고, 이동·추적·공격은 소환체가 실행합니다.

프리팹: `NetworkObject` + `SimpleDroneSummon` + `Collider2D` + `NetworkTransform` + 외형

```
SimpleDroneSummon summon = SpawnOwnedEntity(
    summonPrefab, context.Action, context.AimWorldPosition, maxCount: 1);
return summon != null;
```

```
public sealed class SimpleDroneSummon : CharacterSummon
{
    [SerializeField] private float followSpeed = 3f;
    [SerializeField] private float followDistance = 1.5f;
    [SerializeField] private float attackInterval = 1f;
    [SerializeField] private float attackRadius = 4f;
    [SerializeField] private float damage = 8f;
    [SerializeField] private LayerMask targetLayer;

    public override void FixedUpdateNetwork()
    {
        base.FixedUpdateNetwork();
        if (!Object.HasStateAuthority || !IsActive || IsDestroying)
            return;

        CharacterBase owner = OwnerCharacter;
        if (owner == null)
            return;

        Vector2 ownerPosition = owner.transform.position;
        if (Vector2.Distance(transform.position, ownerPosition) > followDistance)
            transform.position = Vector2.MoveTowards(
                transform.position, ownerPosition, followSpeed * Runner.DeltaTime);

        if (!TryUseInterval(attackInterval))
            return;

        IDamageable target = FindFirstDamageableInCircle(
            transform.position, attackRadius, targetLayer);
        if (target != null)
            DealDamage(target, damage);
    }
}
```

State Authority만 이동과 공격을 결정하고, `NetworkTransform`이 위치를 다른 클라이언트에 동기화합니다.

16. 물리 투척체 생성과 퓨즈

직선 총알은 [CharacterProjectile](#), 중력과 충돌이 필요한 수류탄·섬광탄·연막탄은 [CharacterThrowable](#)을 사용합니다.

```
MyFlashThrowable flash = SpawnThrowable(
    flashPrefab,
    context.Action,
    ProjectileOrigin.position,
    context.AimDirection,
    throwSpeed,
    maxCount: 1);

return flash != null;
```

프리팸 필수 구성

컴포넌트	설정
NetworkObject	필수
CharacterThrowable 파생 스크립트	필수
Rigidbody2D	Dynamic
Collider2D	Trigger 해제
NetworkRigidbody2D	NetworkTransform과 함께 사용하지 않음

퓨즈 시작 방식

방식	시작 시점
OnSpawn	생성 직후
OnGroundContact	Ground Layer에 처음 접촉한 뒤
Manual	StartThrowableFuse 호출 시

[Fuse Seconds](#)가 지나면 [OnFuseExpiredAuthority](#)가 한 번 호출됩니다. 폭발·섬광·연막 효과는 투척체 파생 클래스에서 구현합니다.

17. 생성한 오브젝트 조회·제거·개수 제한

같은 스킬로 만든 오브젝트는 행동 종류로 조회하거나 한 번에 제거할 수 있습니다.

```
IReadOnlyList<SparkQNode> nodes =
    GetOwnedEntities<SparkQNode>(CharacterActionType.SkillQ);

if (nodes.Count > 0)
    DestroyOwnedEntity(nodes[0]);

DestroyOwnedEntities(CharacterActionType.SkillQ);
```

개수 제한

설정	결과
maxCount: 2	동시에 최대 2개 유지
replaceOldest: true	제한 초과 시 가장 오래된 오브젝트 교체
replaceOldest: false	제한에 도달하면 새 생성 실패

상세 생성 함수가 필요한 경우

- 한 행동에서 서로 다른 오브젝트 그룹을 따로 관리할 때
- 세부 초과 정책이나 생성 실패 이유가 필요할 때

이 경우에만 `OwnedEntityGroupId`, `OwnedEntitySpawnRequest`, `OwnedEntitySpawnResult`를 사용합니다.

18. HP·피해·소유자 이탈·후속 처리

생존과 소유자 처리

항목	설정과 결과
Manual	스킬 또는 정책으로 제거
Health	HP가 0이 되면 제거
Duration	제한시간이 끝나면 제거
HealthOrDuration	HP 소진과 시간 만료 중 먼저 발생한 조건
Owner Exit: Destroy	소유자 사망·퇴장 시 제거
Owner Exit: ExpireNormally	남은 제한시간 또는 자동 튜즈까지 유지

피해와 권한

- 기본값은 적 피해 허용, 자가 피해와 아군 피해 차단입니다.
- HP 변경, 이동, 공격과 제거는 State Authority에서 한 번만 처리합니다.
- 생성과 제거는 Fusion NetworkObject 상태로 모든 클라이언트에 동기화됩니다.

제거 사유와 후속 콜백

함수/정보	사용 시점
DestroyReason	HP 소진, 시간 만료, 개수 초과, 소유자 이탈 등 제거 원인 확인
OnOwnedEntityDestroyed	프리팸 내부 파괴 후속 처리
OnOwnedEntityDestroyedRendered	원격 클라이언트 파괴 연출
OnOwnedEntityDamageDealt	소유 캐릭터가 실제 적용 피해 확인

캐릭터 스크립트에서는 `Runner.Spawn`과 `Runner.Despawn`을 직접 호출하지 않습니다.

19. 타이머와 수명 주기

지연 실행

```
CharacterTimerHandle reloadTimer = ScheduleTimer(1.5f, () =>
{
    AddActionCharges(CharacterActionType.SkillQ, 1);
});

CancelTimer(reloadTimer);
```

Coroutine, **Invoke**, **Time.time** 대신 네트워크 Simulation 틱에서 실행되는 공통 타이머를 사용한다.

캐릭터 수명 주기 훅

훅	작업
OnCharacterSpawned	잔탄, 모드, 캐릭터별 초기 상태
OnCharacterDespawned	외부 이벤트 구독 해제, 런타임 참조 정리
OnResetCharacter	변신, 연속기, 충전, 조준 모드 초기화
OnDied	사망 순간 캐릭터별 처리
OnHealthChanged	HUD 외 캐릭터별 상태 반응

공통 Reset은 체력, 쿨다운, 잔탄, 타이머, 슬로우, 히트스톤, 이동 잠금과 궁극기 게이지를 초기화한다. 캐릭터가 별도로 만든 필드는 **OnResetCharacter**에서 초기화한다.

20. 자동 검사 도구와 Fusion 등록

Project 창에서 캐릭터 또는 캐릭터 소유 오브젝트 프리팹을 선택한다.

Tools > Project MS > Character > Validate Selected Character Asset

캐릭터 검사

- CharacterBase 파생 컴포넌트
- NetworkObject
- Rigidbody2D
- Collider2D
- NetworkTRSP 계열 위치 동기화
- CharacterDefinition 연결
- CharacterVisualController 존재

투척체 검사

- NetworkObject, Rigidbody2D, Collider2D, NetworkRigidbody2D
- Rigidbody2D가 움직이는 물리 방식(Dynamic)
- Collider2D의 통과형 충돌(Is Trigger)이 꺼져 있음
- 위치 동기화 컴포넌트가 정확히 1개
- OnGroundContact의 Ground Layer

자동 검사 도구(Validator) 통과만으로 끝난 것은 아니다. 모든 네트워크 프리팹이 Fusion의 네트워크 프리팹 목록(Prefab Table)에 등록되어 생성 가능한지도 확인한다.

21. 테스트 순서

단일 기능 테스트

1. 입력 1회에 행동 1회
2. 성공은 true, 실패는 false
3. 잔탄과 쿨다운 변화
4. 판정 위치와 Layer
5. 실제 적용 데미지와 넉백
6. VFX, SFX, Animator Trigger

Fusion 2인 Shared 테스트

1. Host와 Client가 자기 캐릭터만 조작한다.
2. 이동, 점프, 대시, 조준 방향이 양쪽에서 같다.
3. 공격, 투사체, 설치물 생성이 한 번만 발생한다.
4. 캐릭터와 캐릭터 소유 오브젝트의 HP가 양쪽에서 같다.
5. Owner 사망/이탈 시 정책대로 제거된다.
6. 제거 VFX와 SFX가 양쪽에서 한 번씩 보인다.
7. 라운드 Reset 뒤 체력, 쿨다운, 잔탄, 모드가 초기화된다.

고속 투사체

벽과 얇은 Collider를 통과하지 않는지, 관통탄이 같은 대상을 중복 타격하지 않는지 확인한다.

22. 기능별 레시피

기능	조합
검 베기	Arc 조희 + DealDamage
폭발/오라	Circle 조희 + DealDamage
레이저/관통 직선	Line 조희 또는 Pierce Projectile
기본 총알	CharacterProjectile + SpawnProjectile
함정	CharacterDeployable + HealthOrDuration
지속 장판	CharacterDeployable + Duration + 주기 효과 컴포넌트
포탑	CharacterDeployable + 감지 + 공격 컴포넌트
드론/소환수	CharacterSummon + 이동/대상/공격 모듈
수류탄	CharacterThrowable + 지면 충돌 시 대기시간 시작(OnGroundContact)
원격 기폭 폭탄	CharacterThrowable + 코드 호출 시 대기시간 시작(Manual)
재장전	Charges + AutoCooldown false + ScheduleTimer
후방 공격 보너스	IsBehindTarget + ModifyOutgoingDamage
게이지 궁극기	Definition Ultimate Gauge
변신/모드	Action enable/disable + OnResetCharacter 정리

새 기능은 표의 공통 조합으로 먼저 설계하고, 부족한 사용 규칙만 공통 기반 코드 담당자와 확장한다.

23. 금지 패턴과 대체 API

금지 패턴	문제	대체
캐릭터에서 Runner.Spawn	소유권, 등록, 콜백 누락	SpawnProjectile/SpawnOwnedEntity/SpawnThrowable
캐릭터에서 [Networked] , [Rpc] 부터 추가	공통 사용 규칙 우회	공통 기반 코드 확장 검토
대상 HP 직접 수정	권한과 실제 적용량 불일치	DealDamage/DamageRequest
Runner.Despawn 직접 호출	제거 사유/VFX/등록 해제 누락	DespawnProjectile/DestroyOwnedEntity
Update 에서 패시브 판정	클라이언트별 결과 차이	OnPassiveTick/이벤트 훅
Coroutine/Invoke로 전투 타이머	네트워크 틱 불일치	ScheduleTimer
소유물 썬 검색	소유자와 그룹 혼동	GetOwnedEntities
스킬 이름 대신 Q/E로 최종 명명	입력 변경 시 의미 상실	역할 기반 이름
연결형 프리팹에서 원본에 모두 적용 (Apply All to Base)	다른 캐릭터 프리팹까지 변경	공통 변경을 별도로 검토

24. 문제 해결

증상	먼저 확인
캐릭터가 비활성화됨	Definition 또는 Visual Profile 누락
점프가 안 됨	Definition Ground Layer, Collider, 지면 Layer
투사체 생성 실패	NetworkObject, 위치 동기화 컴포넌트 수, 상태 변경 권한(State Authority)
투사체가 벽 관통	Wall Layer, Physics Matrix, Collider 크기, Skin Width
투척체 생성 실패	움직이는 Rigidbody2D, 통과형이 아닌 Collider2D, NetworkRigidbody2D, 대기시간용 Ground Layer
설치물 생성 실패	Group ID, maxCount, Overflow/OwnerExit 정책
데미지가 두 번 적용됨	직접 충돌 코드와 공통 DealDamage가 동시에 실행되는지
VFX가 두 번 보임	로컬 Instantiate와 공통 RPC 효과를 동시에 호출하는지
원격 캐릭터가 내 마우스를 따라감	로컬 Update에서 원격 Transform을 수정하는지
Reset 뒤 모드가 남음	OnResetCharacter에서 캐릭터별 필드 초기화
소유자 이탈 후 오브젝트가 남음	OwnerExitPolicy와 종료 조건

25. 최종 체크리스트

파일과 이름

- [] 클래스, 파일, 프리팹, 데이터 이름이 표준을 따른다.
- [] 캐릭터 전용 파일이 캐릭터 폴더에 모여 있다.
- [] 기존 캐릭터 복사본이 아니라 CharacterTemplate을 원본으로 한 연결형 프리팹이다.

캐릭터 프리팹

- [] 루트 필수 컴포넌트와 위치 동기화가 정확히 1개다.
- [] 수치 설정 파일(Definition), 화면 표현 제어기(Visual), 비주얼 설정(Profile), 위치 기준점(Sockets)이 연결됐다.
- [] Collider와 Body가 실제 아트 크기에 맞는다.
- [] CharacterTemplate 컴포넌트가 제거됐다.

전투 기능

- [] 행동 성공/실패 반환값이 정확하다.
- [] 데미지, 투사체, 캐릭터 소유 오브젝트가 공통 API를 사용한다.
- [] 잔탄, 쿨다운, 게이지, Reset이 정상이다.
- [] 패시브가 적절한 훅에 있다.

네트워크

- [] 자동 검사 도구(Validator)를 통과했다.
- [] Fusion Prefab 등록을 확인했다.
- [] Host/Client에서 생성, 데미지, 제거가 한 번씩 발생한다.
- [] 소유자 사망, 이탈, 라운드 Reset을 확인했다.

부록 A. 공통 API 빠른 찾기

영역	API/상태
행동	SetActionEnabled, SetActionCharges, AddActionCharges
쿨다운	SetCooldownDuration, SetAutoCooldown, StartCooldown, ClearCooldown
이동	StartDash, SetMovementEnabled, ApplySlow
방향	AimDirection, FacingDirection, IsBehindTarget
체력	CurrentHealth, MaxHealth, Heal, DealDamage
판정	FindDamageablesInCircle/Box/Line/Arc
투사체	SpawnProjectile, DespawnProjectile
캐릭터 소유 오브젝트	SpawnOwnedEntity, GetOwnedEntities, DestroyOwnedEntity
투척체	SpawnThrowable, StartThrowableFuse
타이머	ScheduleTimer, CancelTimer
궁극기	IsUltimateGaugeMode, UltimateGaugeCurrent, UltimateGaugeMax
수명 주기	OnCharacterSpawned, OnCharacterDespawned, OnResetCharacter
결과 혹	OnDamaged, OnDamageDealt, OnDamageableDealt, OnProjectileDespawned

공통 API로 해결되지 않는 기능은 캐릭터 코드에 Fusion 구현을 추가하기 전에 공통 기반 코드 담당자와 사용 규칙을 먼저 정의한다.

부록 B. 입력·봉인·조준 API

캐릭터 스크립트에서는 아래 protected 공통 API만 사용합니다. 네트워크 필드나 RPC를 캐릭터별로 다시 만들지 않습니다.

입력·봉인·조준 API

목적	API
버튼 입력 1회 감지	WasInputPressed(target, CharacterInputType...)
점프 유지 상태	IsInputHeld(target, CharacterInputType.Jump)
좌우 이동 입력	GetObservedMoveDirection(target)
이동·점프·공격·스킬 봉인	ApplyControlSeal(target, controls, seconds)
실제 조준 반전	ApplyAimInversion(target, seconds)
실제 조준 각도 변경	ApplyAimAngleOffset(target, degrees, seconds)
조준선 UI 위치 변경	SetCrosshairOffset(target, pixels, seconds)
조준선 UI 표시·숨김	SetCrosshairVisible(target, visible, seconds)
기본 커서 표시·숨김	SetSystemCursorVisible(target, visible, seconds)
기본 커서를 조준선으로 교체	ReplaceCursorWithCrosshair(target, seconds)

자주 쓰는 조합

```
// 상대의 모든 공격과 스킬 봉인
ApplyControlSeal(target, CharacterControlType.AllActions, 2f);

// 조준 반전 + 조준선 교체
ApplyAimInversion(target, 3f);
ReplaceCursorWithCrosshair(target, 3f);

// 조준선만 오른쪽 80픽셀 이동
SetCrosshairOffset(target, new Vector2(80f, 0f), 3f);
```

상세 설명과 입력 감지 예제는 8.1~8.3을 확인합니다.